

Ericsson India Private Limited

SECURETASK

Security-Focused Full-Stack Task Management Application

Technical Project & Security Assessment Report

Prepared By

Parthak Bhardwaj

Under the Guidance of

Supriya Baig and Sakshi

Project Duration

8 Weeks

Internship Location

Ericsson, Noida, Uttar Pradesh, India

Contents

1	Executive Summary	1
2	Project Overview	1
2.1	Objectives	1
2.2	Main Features	1
3	Technology Stack	2
4	System Architecture	2
4.1	Frontend Layer	2
4.2	Backend Layer	2
4.3	Database Layer	2
5	Authentication and Authorization	3
5.1	Authentication Flow	3
5.2	Authorization	3
5.3	Password Security	3
6	Security Implementation	3
6.1	JWT Secret Management	3
6.2	CORS Security	3
6.3	Input Validation	4
7	Security Testing Methodology	4
7.1	Manual Security Tests	4
8	Security Tools and Results	4
9	Vulnerabilities and Remediation	4
10	OWASP ZAP Assessment	4
10.1	Bcrypt Password-Length Finding	6
11	Screenshots and Security Evidence	7
11.1	Application Screens	7
11.2	Security Tool Evidence	8
11.3	CORS Verification	10
11.4	OWASP ZAP Evidence	11
12	Final Security Test Summary	13

13 Deployment	13
14 Conclusion	13
15 Project Links	14

List of Figures

1	High-level application architecture	2
2	SecureTask login screen	7
3	SecureTask dashboard	7
4	SecureTask task management interface	8
5	Bandit final security scan	8
6	SonarQube final security and code-quality analysis	9
7	Gitleaks repository secret scan	9
8	OWASP Dependency-Check final scan	10
9	Trusted origin CORS verification	10
10	Untrusted origin CORS verification	11
11	OWASP ZAP alert overview	11
12	Manual root-cause analysis of the ZAP password finding	12
13	Bcrypt password-length validation after remediation	12
14	Production request flow	13

1 Executive Summary

SecureTask is a security-focused full-stack task management application developed during an eight-week internship project. It provides user registration and login, JWT-based authentication and authorization, task CRUD operations, dashboard statistics, search and filtering, and a user profile.

The application is built with React, Flask, PostgreSQL, SQLAlchemy, JWT and bcrypt, and is deployed on Vercel, Render and Neon PostgreSQL.

Security was addressed throughout the project through manual testing, static analysis, secret scanning, dependency analysis, code-quality analysis and dynamic testing, using Bandit, Semgrep OSS, Gitleaks, OWASP Dependency-Check, pip-audit, Safety, SonarQube and OWASP ZAP.

Headline Results

- **5 findings** identified and remediated: hardcoded test credential, Flask debug mode, permissive CORS, a vulnerable dependency, and missing bcrypt password-length validation.
- **0 findings** in the final Bandit, Semgrep, Gitleaks, Dependency-Check, pip-audit and Safety scans.
- **SonarQube:** 0 security vulnerabilities and Quality Gate passed (41 general issues remain).
- **Not yet done:** a complete post-remediation OWASP ZAP active scan.

2 Project Overview

2.1 Objectives

- Develop a complete full-stack task management application.
- Implement secure authentication and authorization, with passwords protected by bcrypt and sessions handled by JWT.
- Implement task CRUD functionality.
- Perform manual and automated security testing.
- Identify and remediate security issues.
- Deploy the application and document the security assessment.

2.2 Main Features

- User registration and login
- JWT authentication and protected routes
- User-level authorization on all task operations
- Task create, read, update and delete
- Task search and category filtering
- Dashboard statistics and user profile
- Responsive user interface

3 Technology Stack

Category	Technologies
Frontend	React.js, JavaScript, Tailwind CSS, Vite, React Router
HTTP Client	Axios
Backend	Python, Flask
Database	PostgreSQL, SQLAlchemy, psycopg2-binary
Authentication	JWT, Flask-JWT-Extended
Password Security	bcrypt
Cloud Services	Cloudinary
Version Control	Git, GitHub
Frontend Deployment	Vercel
Backend Deployment	Render
Database Hosting	Neon PostgreSQL
Security Testing	Bandit, Semgrep OSS, Gitleaks, OWASP Dependency-Check, pip-audit, Safety, SonarQube, OWASP ZAP

Table 1: SecureTask technology stack

4 System Architecture

SecureTask follows a client–server architecture with a clear separation between the presentation, application and data layers.



Figure 1: High-level application architecture

4.1 Frontend Layer

The frontend handles the user interface, authentication screens, task management, dashboard, profile, search and filtering, protected routes and all API communication.

4.2 Backend Layer

The Flask backend exposes REST APIs for user registration, authentication, JWT validation, task management, authorization and database operations.

4.3 Database Layer

PostgreSQL stores application data, while SQLAlchemy provides the ORM layer between the Flask application and the database.

5 Authentication and Authorization

5.1 Authentication Flow

SecureTask uses JWT-based authentication with Flask-JWT-Extended.

1. The user registers an account.
2. The password is hashed with bcrypt and the hash is stored in PostgreSQL.
3. The user logs in with their credentials.
4. bcrypt verifies the password against the stored hash.
5. The backend generates a JWT.
6. The JWT is sent with every protected API request.

Authenticated requests use the **Authorization** header, and the backend validates the token before granting access to protected endpoints:

```
Authorization: Bearer <JWT>
```

5.2 Authorization

Authorization determines what an authenticated user may access. SecureTask enforces it through:

- Protected Flask API routes and JWT validation
- User-specific resource checks on task operations
- React protected routes on the client

Together these prevent users from reaching resources that belong to other users through unauthorized requests.

5.3 Password Security

Passwords are never stored as plaintext. bcrypt hashes passwords at registration and verifies them at login.

6 Security Implementation ---

6.1 JWT Secret Management

Sensitive configuration, including the JWT secret, is supplied through environment variables rather than hardcoded in the source code.

6.2 CORS Security

An explicit allowlist of trusted origins replaces the earlier permissive configuration:

```
ALLOWED_ORIGINS = [  
    "https://secure-task-chi.vercel.app",  
    "http://localhost:5173",  
    "http://127.0.0.1:5173"  
]  
  
CORS(  
    allow_origin_regex=r"^(https://secure-task-chi.vercel.app|http://localhost:5173|http://127.0.0.1:5173)$"
```

```
    app,  
    resources={r"/*": {"origins": ALLOWED_ORIGINS}}  
)
```

Both trusted and untrusted origins were tested to verify the behavior (see Figures 9 and 10).

6.3 Input Validation

Validation was added for security-sensitive inputs. A specific check was implemented for passwords because bcrypt only processes the first 72 bytes of its input.

7 Security Testing Methodology ---

The assessment combined several complementary approaches:

- Manual application security testing
- Static Application Security Testing (SAST)
- Secret scanning
- Dependency vulnerability scanning
- Code-quality and security analysis
- Dynamic Application Security Testing (DAST)

7.1 Manual Security Tests

The following scenarios were tested manually:

- SQL injection
- Cross-site scripting (XSS)
- Authentication bypass
- IDOR / authorization
- Password hashing
- Invalid task IDs
- Malformed JSON
- Verbose error handling

Directory traversal and file upload testing were considered not applicable because the application does not implement that functionality.

8 Security Tools and Results ---

9 Vulnerabilities and Remediation ---

10 OWASP ZAP Assessment ---

OWASP ZAP was used for dynamic testing against the local application:

Tool	Result / Purpose
Bandit	Python security static analysis. Final: High 0, Medium 0, Low 0.
Semgrep OSS	459 rules, 46 targets, 0 findings.
Gitleaks	17 commits scanned, 0 leaks.
OWASP Dependency-Check	150 dependencies scanned; 0 vulnerable dependencies after remediation.
pip-audit	No known vulnerabilities in the specified requirements file.
Safety	0 vulnerabilities in the requirements-file check.
SonarQube	0 security vulnerabilities; Quality Gate passed; 41 total issues remain.
OWASP ZAP	3,465 requests, 86 alert instances, 19 distinct alert types. Findings were manually triaged.

Table 2: Security assessment results by tool

Finding	Remediation
Hardcoded test secret	Removed the secret and moved sensitive configuration to environment variables.
Flask debug configuration	Debug mode disabled.
Permissive CORS	Replaced with an explicit trusted-origin allowlist.
Vulnerable <code>nanoid</code> dependency	Updated from <code>3.3.17</code> to <code>3.3.18</code> .
Bcrypt password-length issue	Added UTF-8 byte-length validation before passwords reach bcrypt.

Table 3: Security findings and remediation

- Frontend: `http://localhost:5173`
- Backend: `http://127.0.0.1:5000`

The active scan generated 3,465 requests, 86 alert instances and 19 distinct alert types. Alert instances are not equivalent to confirmed unique vulnerabilities, so each was manually triaged.

10.1 Bcrypt Password-Length Finding

ZAP reported password-related observations classified as *Buffer Overflow* and *Format String Error*. Manual reproduction with an oversized password produced an HTTP 500 response. Investigation traced this to bcrypt's 72-byte password limit rather than to a memory-safety flaw.

The application was updated to validate the password byte length:

```
if len(password.encode("utf-8")) > 72:
    return jsonify({
        "message": "Password must be 72 bytes or fewer"
    }), 400
```

After remediation, the same oversized password returned HTTP 400 without exposing a traceback.

Conclusion of the Investigation

The ZAP alert labels were treated as scanner classifications. The confirmed root cause was missing bcrypt input validation, now fixed and verified.

11 Screenshots and Security Evidence

This section presents selected screenshots of the application and of the main security activities. The complete evidence set is maintained in:

```
docs/security/evidence/
```

11.1 Application Screens

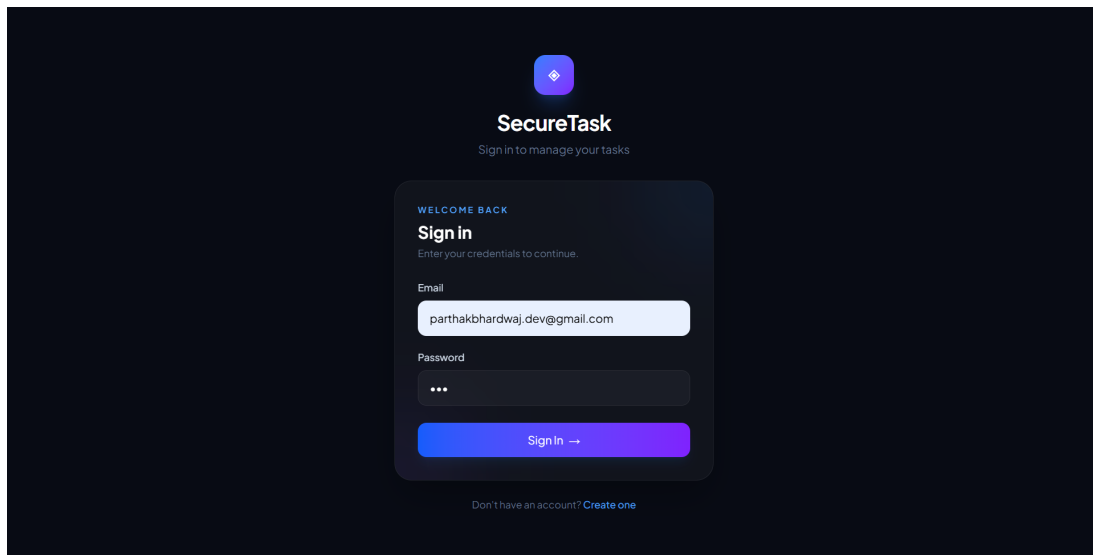


Figure 2: SecureTask login screen

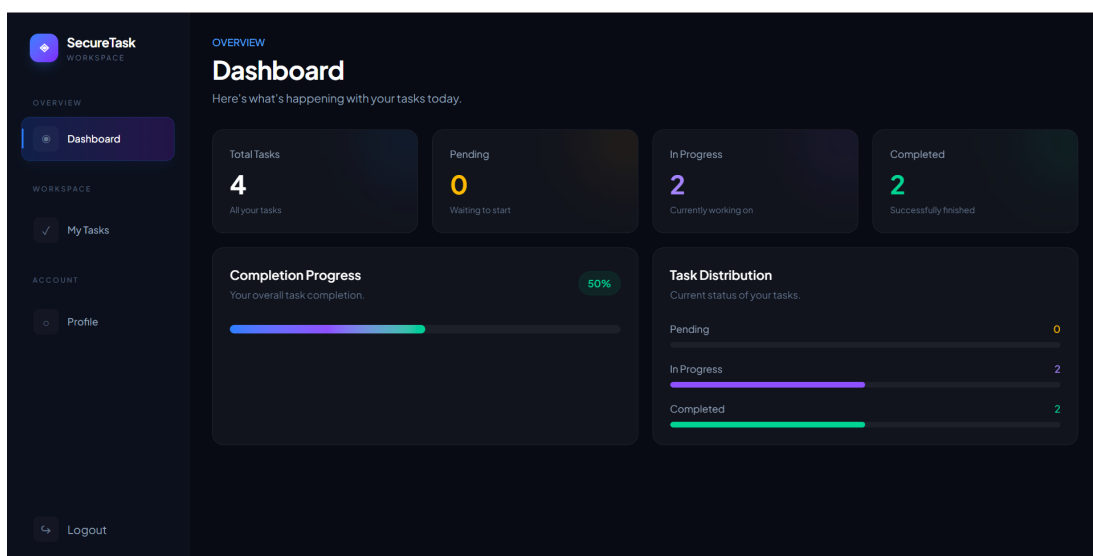


Figure 3: SecureTask dashboard

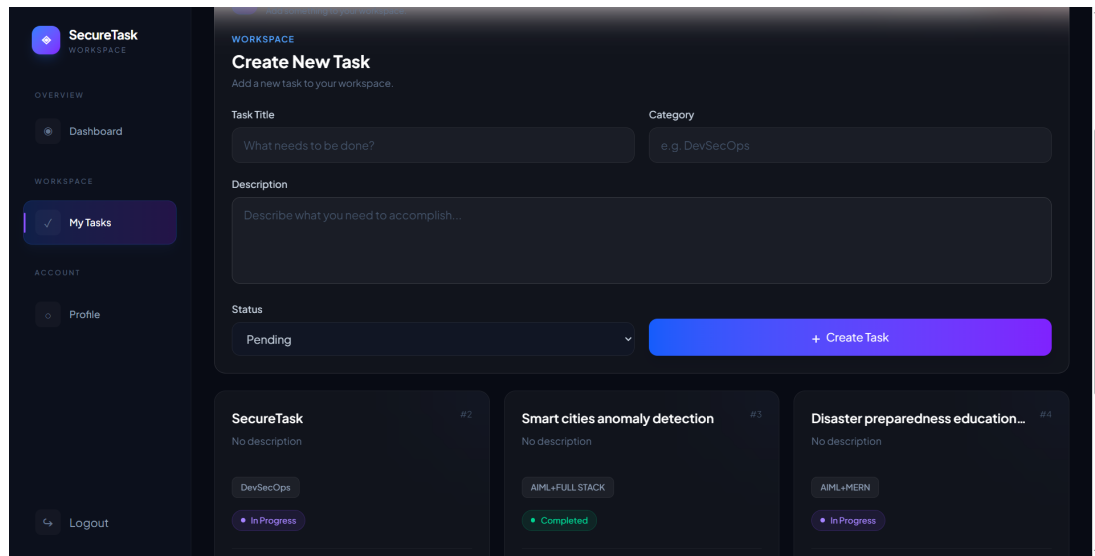


Figure 4: SecureTask task management interface

11.2 Security Tool Evidence

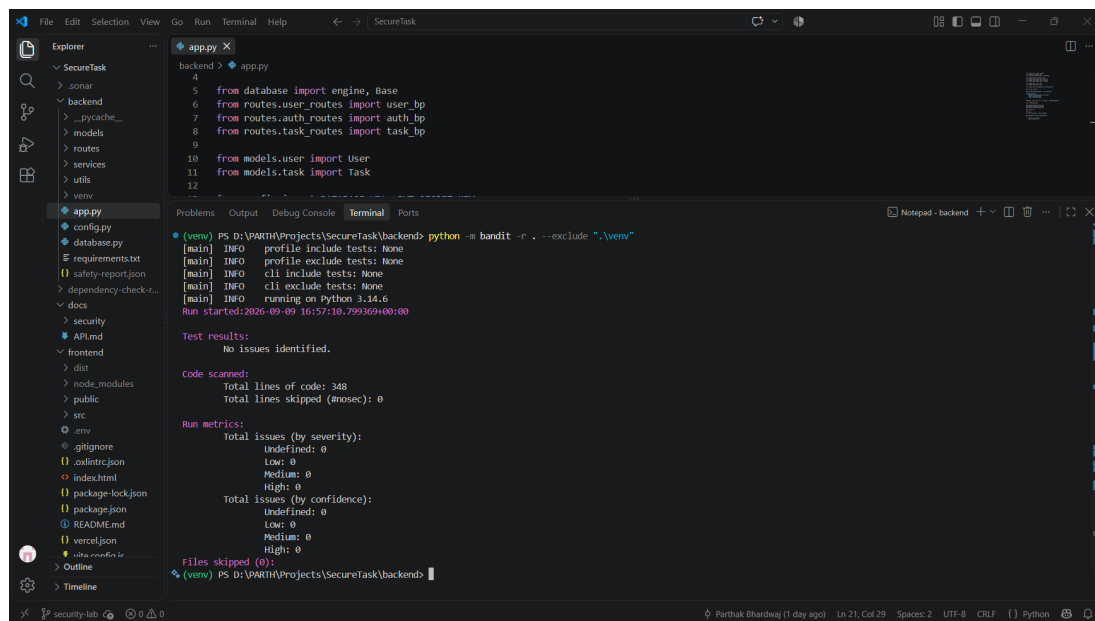


Figure 5: Bandit final security scan

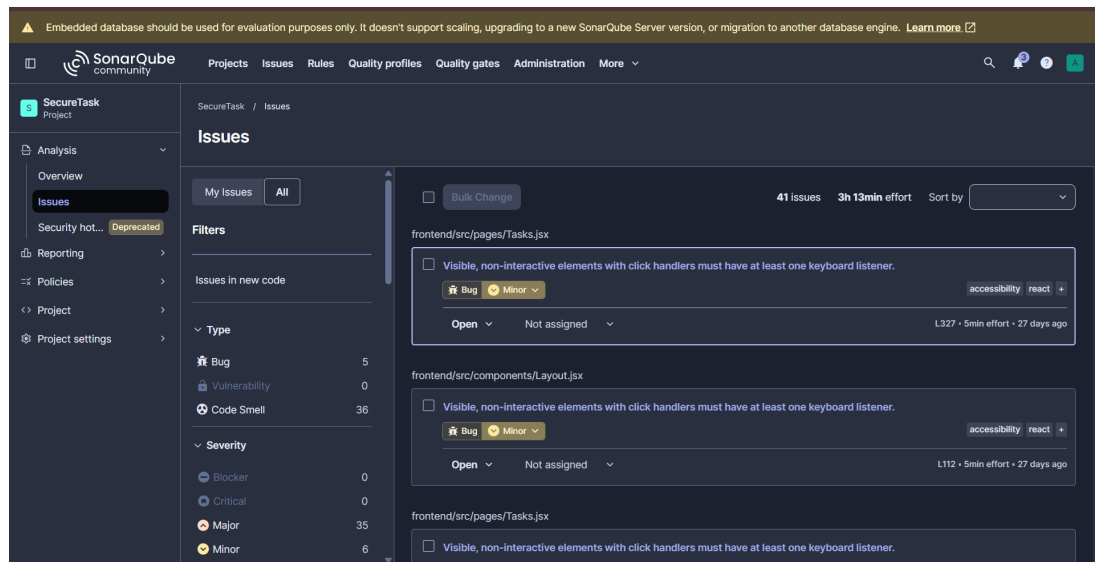


Figure 6: SonarQube final security and code-quality analysis

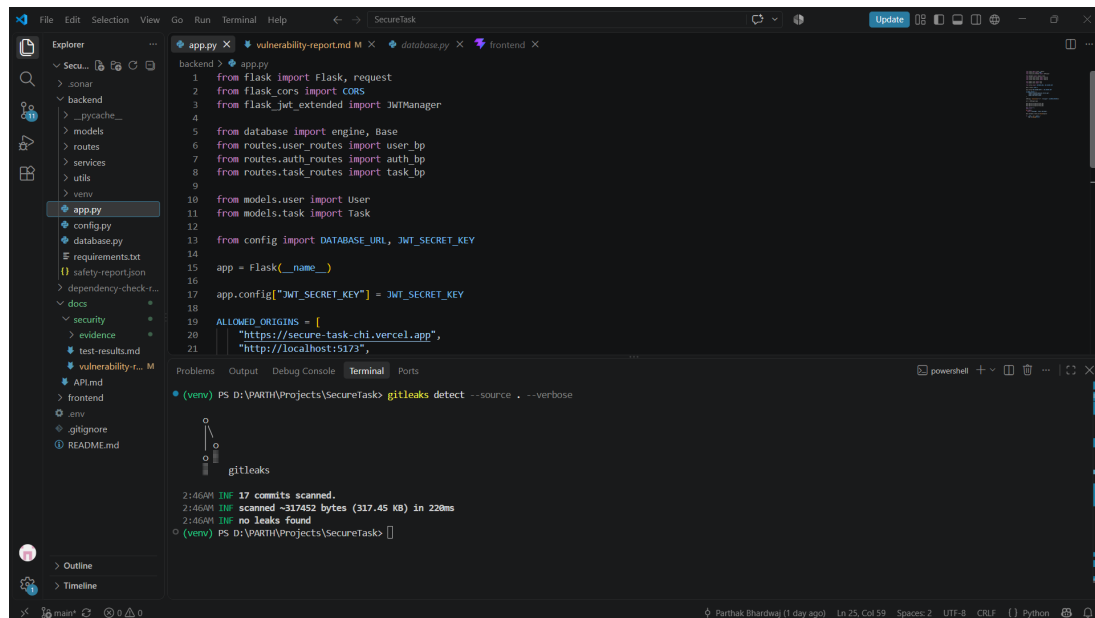


Figure 7: Gitleaks repository secret scan

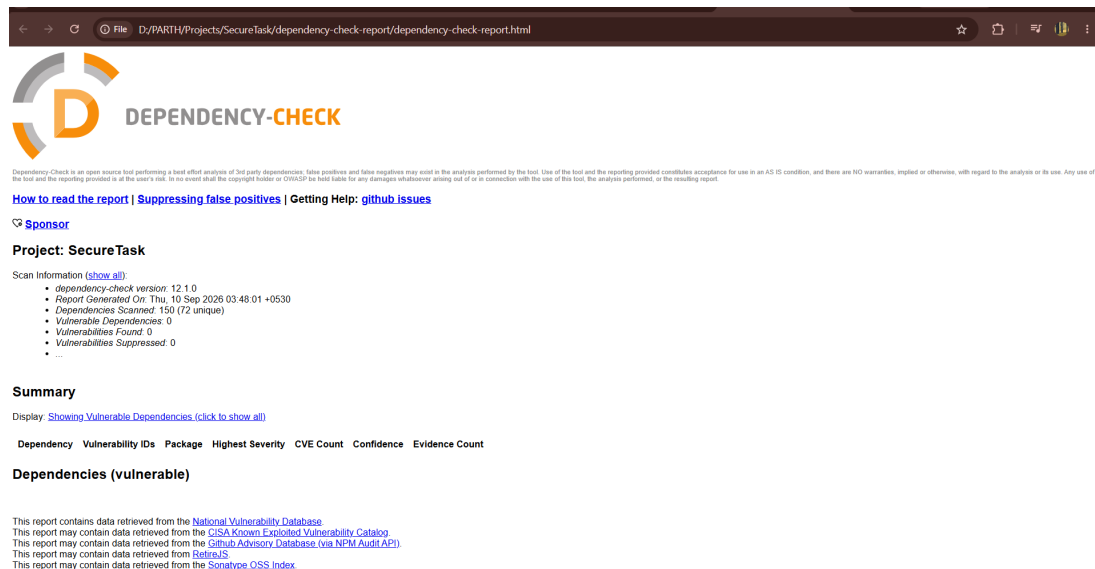


Figure 8: OWASP Dependency-Check final scan

11.3 CORS Verification

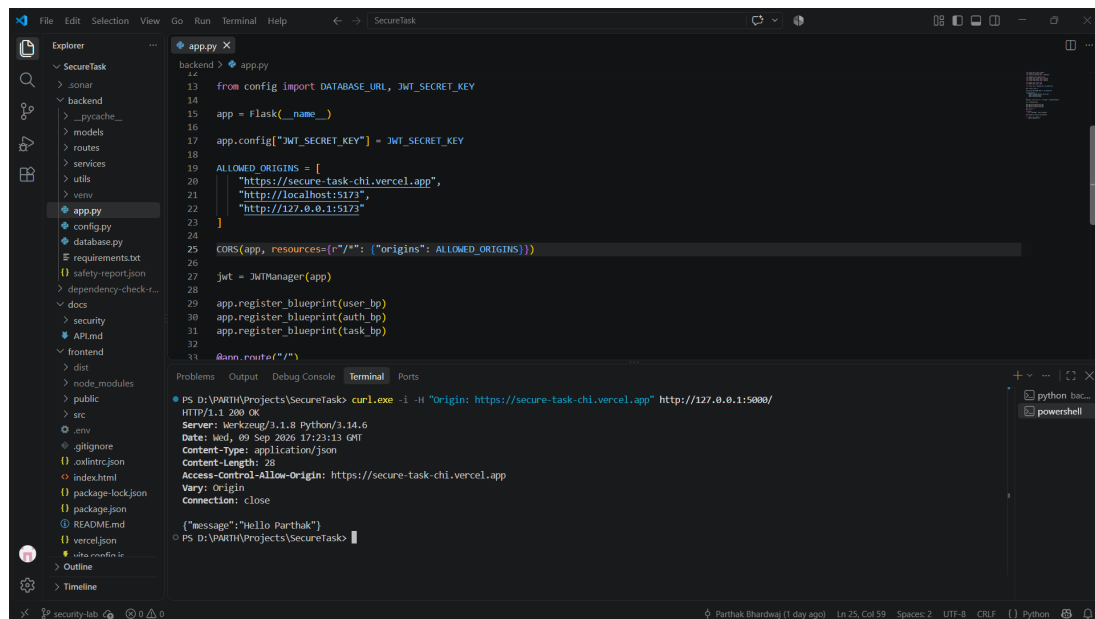


Figure 9: Trusted origin CORS verification

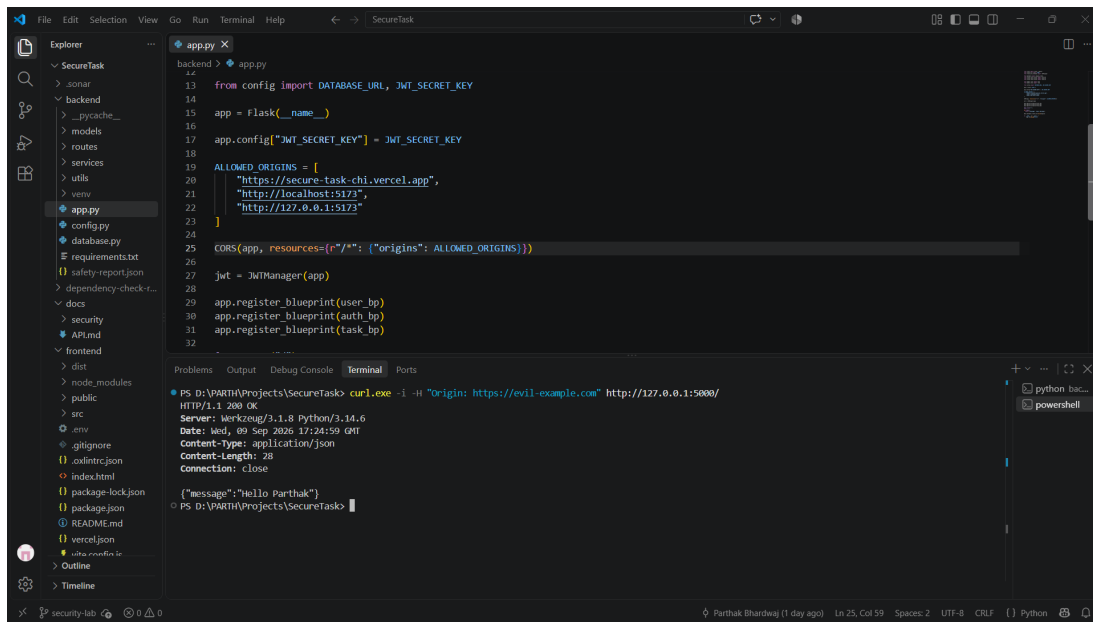


Figure 10: Untrusted origin CORS verification

11.4 OWASP ZAP Evidence

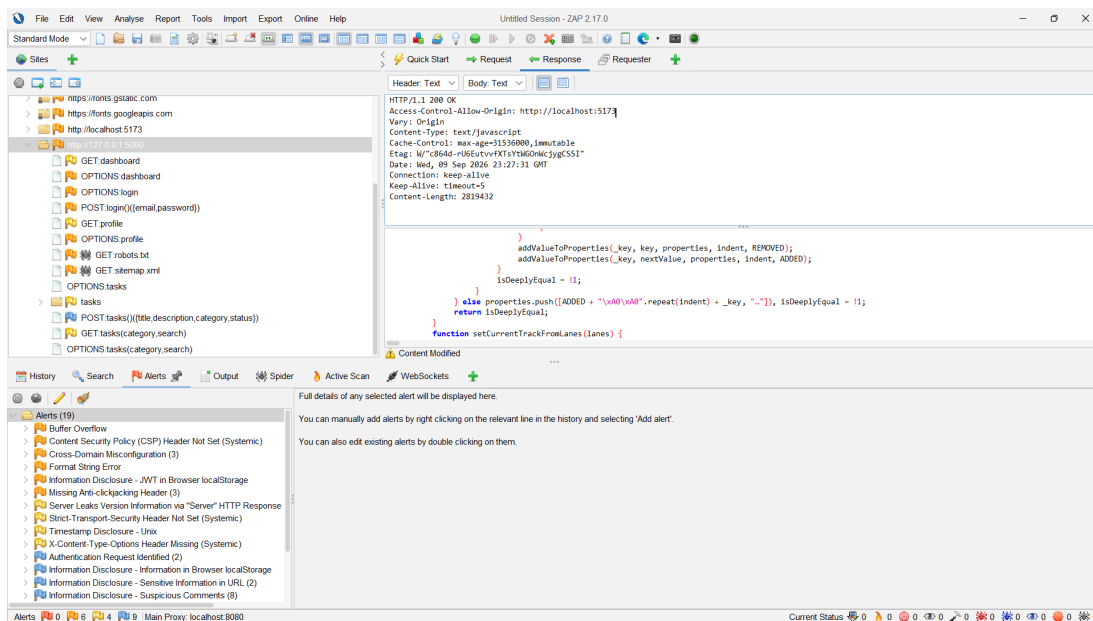
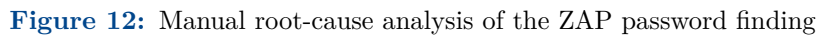


Figure 11: OWASP ZAP alert overview



12 Final Security Test Summary

Test / Check	Status
SQL injection	PASS
XSS	PASS
Authentication bypass	PASS
IDOR / authorization	PASS
Password hashing	PASS
Invalid task ID	PASS
Malformed JSON	PASS
Verbose error handling	PASS
Bandit	PASS
Semgrep	PASS
Gitleaks	PASS
Dependency-Check	PASS
pip-audit	PASS
Safety requirements check	PASS
SonarQube	PASS (with caveats)
Bcrypt remediation	PASS
Full post-fix ZAP scan	NOT PERFORMED

Table 4: Final security testing summary

13 Deployment

Frontend	https://secure-task-chi.vercel.app
Backend	https://securetask-backend.onrender.com
Database	Neon PostgreSQL

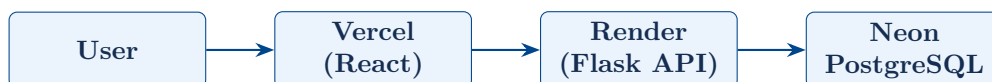


Figure 14: Production request flow

14 Conclusion

SecureTask combines full-stack development with a structured security assessment. The project delivered a React and Flask application backed by PostgreSQL, with JWT authentication, bcrypt password protection, authorization, protected routes and a live deployment.

Manual techniques and eight automated tools were used to find issues, and several were remediated and re-verified: insecure CORS, a vulnerable dependency, a hardcoded test secret, debug configuration and missing bcrypt length validation.

The project shows the value of pairing automated scanning with manual investigation, remediation and verification, as the ZAP bcrpt finding illustrates.

15 Project Links

GitHub	https://github.com/bhardwajparthirl/SecureTask
Frontend	https://secure-task-chi.vercel.app
Backend	https://securetask-backend.onrender.com